

Chapter 20.

Automating Reports & Delivering Insights

Get the number in front of the right person, at the right time, reliably enough that they stop wondering where it came from.

Approved chapter v1

Contents

Chapter 20. Automating Reports & Delivering Insights

Why this matters

In plain English

20.1 The automation ladder

20.2 Map the flow before you automate

20.3 Choosing the delivery tool

20.4 Output formats: what to send

20.5 The report as an email

20.6 Sending mail from code, safely

20.7 Scheduling

20.8 Alerts and exception reports

20.9 Delivering to chat

20.10 Low-code automation

20.11 Making an automation trustworthy

20.12 Recipients and confidentiality

20.13 Documenting and handing over

20.14 Measuring what it saved

Common mistakes and how to spot them

In the real world: the flash that stopped

Tools

The project: Riverstone's Daily Sales Flash

Timed challenge: forty minutes

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 20. Automating Reports & Delivering Insights

Part II — The Analyst

Chapter at a glance

You will learn to: place any report on the automation ladder and decide how far up it should go · map a report's flow and time its manual steps before automating anything · choose between VBA, Apps Script, Python, BI subscriptions, and low-code flows · produce the right output: formatted Excel, PDF, CSV, or the email body itself · build an HTML email with KPI tiles, a table, and an embedded chart that renders in Outlook and Gmail · send mail safely from code with SMTP or a workspace API, with credentials outside the code · schedule with Task Scheduler, cron, or a cloud scheduler, and reason about time zones · design exception reports and alerts that people don't learn to ignore · deliver to Teams, Slack, or WhatsApp · make an automation trustworthy: logging, checks, failure alerts, "no data today", retries, idempotency · manage recipients and confidentiality · document and hand over · measure what it saved.

Before you start: Chapter 18 (pandas and scripts), Chapter 19 (spreadsheet automation), Chapter 16 (BI subscriptions), Chapter 15 (chart design), Chapter 14 (checks on data), and Chapter 13 (the SQL the report runs on).

Time needed: 18–22 hours, spread over two to three weeks.

Tools: Python 3.13 or 3.14 with `pandas`, `matplotlib`, `SQLAlchemy`, a database driver and `python-dotenv`; a mail account you're allowed to send from (SMTP, Microsoft 365, or Google Workspace); Windows Task Scheduler or cron. Optional: Power Automate, n8n, Make, or Zapier.

Practice data: the `riverstone_full` database, and `companion/ch20/daily_flash.py` — the finished Daily Sales Flash, which builds the email in this chapter.

Why this matters

An analysis nobody reads has the same value as an analysis nobody did. Delivery is not the packaging around the work; it's the part where the work turns into a decision.

Most analysts discover this the hard way. The report is right, the chart is clear, and it still fails: it lands at 11 a.m. when the meeting was at 9, or in an attachment nobody opens on a phone, or in an inbox where it looks like the eleven other reports that go unread. Or it arrives faithfully for six months and then quietly stops, and nobody notices for three weeks.

This chapter is about the last mile: getting the number in front of the right person, at the right time, in a form they'll act on, reliably enough that they stop thinking about where it comes from. It's also where an analyst's time comes back. A report that takes ninety minutes a day is 30 working days a

year; automating it buys back a month, and the month is what you spend on the analysis nobody has asked for yet.

The chapter builds one thing end to end: Riverstone's **Daily Sales Flash**, a database query that becomes an email with KPI tiles, a table, and a chart, sent every morning, with an exception alert when something is wrong and a failure alert when the job itself breaks.

In plain English

Think about how a newspaper reaches a doorstep.

Someone writes the story (your analysis). Someone else decides what goes on page one (what the reader needs first). It's printed in a fixed format (the same layout every day, so readers know where to look). It's delivered at the same time every morning, whether or not anyone is awake at the press. And if the van breaks down, somebody knows within minutes, because a hundred people ring up.

An automated report is the same machine:

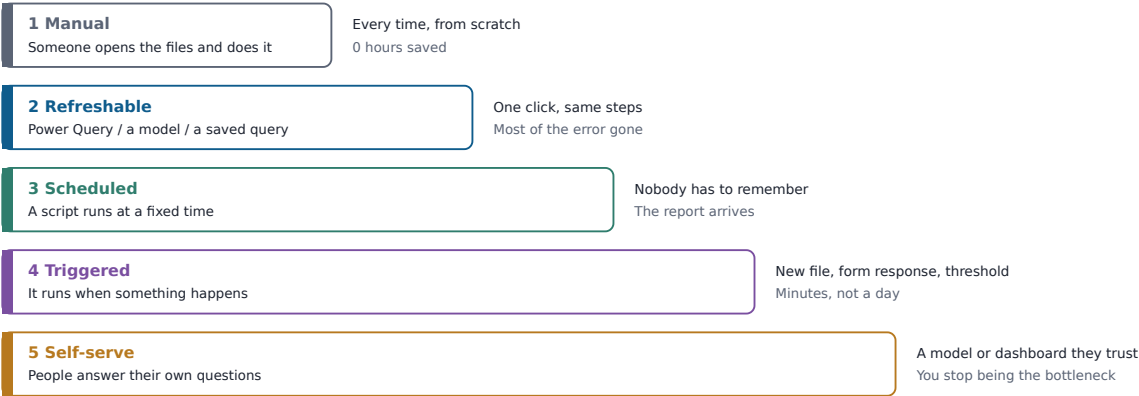
- **The press** is the script: it runs the same way every time.
- **Page one** is your KPI tiles: three or four numbers, in the same place, every day.
- **The delivery round** is scheduling and recipients.
- **The phone calls** are the failure alerts, except you don't wait for the readers to notice. The machine tells you first.

The mistake most first automations make is building a beautiful press with no phone. It works for weeks, then a source file changes name, and the report sends yesterday's numbers, or nothing at all, and everyone keeps trusting it until a decision goes wrong.

20.1 The automation ladder

The analyst's automation ladder

Each rung removes a different kind of work. Most reports should stop at 3; the ones people ask about hourly belong at 5.



Climbing costs effort and adds things that can break: every rung above 2 needs logging, failure alerts, and an owner.

Figure 20.1 – Each rung removes a different kind of work. Most reports should stop at rung 3.

Rung	What it means	Right when
1 Manual	A person does every step	It's truly one-off, or the judgment is the work
2 Refreshable	The steps are recorded: Power Query, a saved query, a model	The shape is stable; the data changes
3 Scheduled	A script runs at a fixed time	The report is regular and the audience is fixed
4 Triggered	It runs when something happens: a file lands, a form is submitted, a threshold breaks	Waiting until tomorrow morning would be too late
5 Self-serve	People answer their own questions in a model or dashboard	The questions vary, and you're the bottleneck

Three practical rules:

1. **Every rung above 2 needs an owner, a log, and a failure alert.** Automation converts a person's attention into a machine's silence; the alert is how you buy the attention back.
2. **Don't skip rung 2.** If the manual steps aren't stable, scheduling them only makes the same mistake happen faster, at 7 a.m., without anyone watching.
3. **Rung 5 is a different job.** Self-serve is a data model and a shared definition (Chapter 16), not a script. It's the right end state for the reports people ask about constantly, and overkill for the rest.

20.2 Map the flow before you automate

Mapping a report's flow, and finding the manual steps

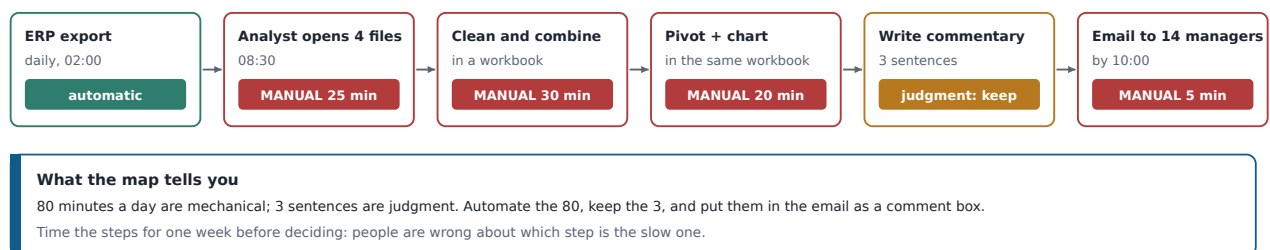


Figure 20.2 — Map every step from source to reader, mark which are manual, and time them for a week.

Before writing any code, draw the report's flow and answer five questions:

1. **Where does the data come from**, and when is it ready? (A report scheduled for 06:00 that reads a table loaded at 06:30 is worse than no report.)
2. **What happens to it**, step by step, and how long does each step take? Time them for a week; people consistently misjudge which step is slow.
3. **Which steps are mechanical and which are judgment?** Automate the mechanical ones. Keep the judgment, and give it a place in the output: a commentary box, three sentences, written by a human.

4. **Who reads it**, on what device, and what decision do they make? A branch manager reading on a phone at 8 a.m. needs four numbers; the finance analyst needs the workbook.
5. **What happens if it's wrong or late?** That tells you how much checking and alerting the automation deserves.

For Riverstone's Daily Flash, the map came out as: ERP loads at 02:00 (automatic) → an analyst opens four files (25 minutes) → cleans and combines (30 minutes) → pivots and charts (20 minutes) → writes three sentences of commentary (judgment) → emails 14 managers (5 minutes). Eighty minutes of mechanical work, three sentences worth keeping.

20.3 Choosing the delivery tool

If the work is...	Use	Because
Inside one workbook, on a desktop	VBA (Chapter 19)	It's already there; formatting, PDF, Outlook
Inside Google Sheets, with forms and triggers	Apps Script (Chapter 19)	Triggers and email are built in, and it runs in the cloud
From a database or API, or needs real transformation	Python (Chapter 18)	Anything else fights you
An existing BI report	Power BI subscriptions and alerts (Chapter 16)	No code, and the model is already governed
Gluing SaaS tools together, owned by the business	Power Automate, n8n, Make, Zapier	Non-developers can maintain them
A pipeline with dependencies and retries	An orchestrator (Chapter 46)	Scheduling is not the same as orchestration

Two decision rules that save arguments:

- **Follow the data.** If the numbers live in a database, the automation belongs where the database is simple to reach, which usually means Python on a server, not a macro on a laptop.
- **Follow the maintainer.** An automation nobody but you can change is a risk. Sometimes the right answer is the slightly worse tool that the team can maintain, and that is a legitimate engineering decision, not a compromise.

Watch out: the laptop trap. Task Scheduler on your own machine is the most common first home for an automation, and it fails when you're on leave, when the laptop sleeps, when IT pushes an update, and when you change jobs. It's fine for a week while you prove the value. Then ask for a server, a VM, a container, or a cloud scheduler.

20.4 Output formats: what to send

Format	Good for	Costs the reader
Email body (HTML)	Daily numbers, alerts, anything read on a phone	Nothing: it's already open
Formatted Excel	Figures people will slice, filter, or paste into a deck	A download and an app
PDF	A fixed record: board packs, statements, anything signed off	Can't be reworked; often unread on phones
CSV	Feeding another system	Useless to humans
A link to a dashboard	Exploration, many questions	A login, and a licence (Chapter 16)

The rule that matters more than the format: **put the answer in the body**. An attachment is a request; a body is an answer. If someone must open a file to learn whether today was good, most of them won't. Send the four numbers in the email and attach the workbook for the two people who want it.

Riverstone's Flash does exactly that: KPI tiles and a table in the body, and (in the project's stretch version) the detailed workbook attached for the finance analyst who reconciles it.

20.5 The report as an email

Email clients are not browsers. Outlook on Windows renders HTML with Microsoft Word's engine; Gmail strips some CSS; phones are 360 pixels wide. What survives everywhere is 2005-era HTML:

- **Tables for layout**, not flexbox or grid, and a fixed width of about 600–640 pixels.
- **Inline styles** (`style="..."` on each element), because `<style>` blocks and external stylesheets are often stripped.
- **Web-safe fonts** (Arial, Segoe UI, Helvetica) with fallbacks; no web fonts.
- **Images with a width attribute and alt text**, because many clients block images by default: the email must still make sense with every image missing.
- **No JavaScript**, ever. It's stripped, and it would be a security problem if it weren't.

Here's the Flash's KPI tile, which is the whole technique in one function: a table cell, a background, inline styles, three lines of text.

```

import os
from pathlib import Path

INK, MUTED, LIGHT, GOOD = "#1d2330", "#5b6475", "#dfe5ec", "#2f7d6d"

def tile(label, value, note="", note_color=MUTED):
    return (
        f'<td style="padding:0 8px 0 0;vertical-align:top">'
        f'<table role="presentation" cellpadding="0" cellspacing="0" width="100%" '
        f'style="background:#f3f6fa;border:1px solid {LIGHT};border-radius:6px">'
        f'<tr><td style="padding:10px 12px">'
        f'<div style="font:12px Arial,sans-serif;color:{MUTED}">{label}</div>'
        f'<div style="font:bold 20px Arial,sans-serif;color:{INK};padding-top:2px">{value}</div>'
        f'<div style="font:12px Arial,sans-serif;color:{note_color};padding-top:2px">{note}</div>'
        f"</td></tr></table></td>")

row = "<table role='presentation'><tr>" + \
    tile("Net revenue", "₹2,511,819", "+2.3% vs last year", GOOD) + \
    tile("Orders", "118", "116 customers") + "</tr></table>"
print(len(row), "characters of HTML")
print(row[:120] + " ...")

```

```

1035 characters of HTML
<table role='presentation'><tr><td style="padding:0 8px 0 0;vertical-align:top"><table
role="presentation" cellpadding=" ...

```

`role="presentation"` tells screen readers that the table is layout, not data. The tiles sit in one row of an outer table, which is how you get a "card row" that works in Outlook.

The chart

A chart in an email is a PNG. Two ways to include it:

- **Base64 data URI** (``): self-contained, no hosting, but Outlook desktop often refuses to show data URIs.
- **CID attachment** (attach the image and reference `cid:chart`): the reliable route for Outlook, and the one to use when the audience is on Microsoft 365.

Either way, generate it with matplotlib exactly as in Chapter 18, keep it under about 620 pixels wide, and write `alt` text that states the finding, because a blocked image should still say something.

```

import base64, io
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6.4, 2.1))
ax.plot([1, 2, 3, 4, 5], [30.6, 33.5, 36.0, 25.1, 26.4], color="#0f5c8c", linewidth=2.2)
ax.spines[["top", "right"]].set_visible(False)
buffer = io.BytesIO()
fig.savefig(buffer, format="png", dpi=150, bbox_inches="tight")
plt.close(fig)

encoded = base64.b64encode(buffer.getvalue()).decode("ascii")
img_tag = f''
print(len(encoded), "base64 characters")
print(img_tag[:80] + " ...")

```

```

29444 base64 characters
Microsoft Graph API</b> ( <code>/users/{id}/sendMail</code> ), app registration with <code>Mail.Send</code> | The supported way for unattended sending; IT approves the app |
| Microsoft 365    | <b>Power Automate</b> flow that sends the mail                                                                 | No app registration; the flow owner sends it                  |
| Google Workspace | <b>Gmail API</b> with a service account, or <b>Apps Script MailApp</b> (Chapter 19)                            | Apps Script is much simpler for Sheets-based flows            |
| Anywhere         | A transactional provider (SendGrid, SES, Postmark)                                                             | Best deliverability for volume; needs domain setup            |

Whichever you use, ask three questions first: **who does the mail appear to come from**, **who can stop it**, and **what happens to bounces**. An automation sending from a personal mailbox to 40 people is a governance problem waiting to be discovered.

## Never send from a laptop as yourself, for long

The honest progression: prove the value with your own mailbox for a week, then move to a service account and a server. The moment the report has more than a handful of recipients, it belongs on a platform that IT knows about.

## 20.7 Scheduling

### Windows Task Scheduler

For a script on a Windows machine or VM:

1. **Task Scheduler** → **Create Task** (not "Basic Task": you need the extra options).
2. **General**: run whether the user is logged on or not; use a service account; tick "Run with highest privileges" only if you truly need it.
3. **Triggers**: daily at 07:00; optionally "repeat every 15 minutes for 1 hour" for retries.
4. **Actions**: program `C:\path\.venv\Scripts\python.exe`, arguments `daily_flash.py 2025-12-18 --send`, and **Start in** the script's folder (the most common cause of "it works when I run it, not when it's scheduled" is a missing working folder).
5. **Settings**: "Stop the task if it runs longer than 1 hour", and "If the task fails, restart every 10 minutes, up to 3 times".

### cron

On Linux or macOS, `crontab -e`:

```
minute hour day month weekday command
0 7 * * * 1-5 cd /opt/riverstone && /opt/riverstone/.venv/bin/python daily_flash.py "$(date +%F)" -
-send >> logs/flash.log 2>&1
```

- **1-5** is Monday to Friday. `0 7 * * *` is every day; `*/15 * * * *` every fifteen minutes.
- `>> logs/flash.log 2>&1` keeps both normal output and errors; without it, cron emails them into a void.
- **cron runs with a minimal environment**: no `PATH` you're used to, no virtual environment. Use absolute paths, and load variables explicitly (`set -a; . /opt/riverstone/.env; set +a`).
- **Escape % as \%** in cron commands, which is why `date +%F` looks odd.

### Cloud schedulers

GitHub Actions (on: `schedule`), Azure Functions timers, AWS EventBridge with Lambda, Google Cloud Scheduler, and the scheduler inside any orchestrator (Chapter 46) all do the same job without a machine you maintain. For a script that runs for a minute a day and needs a database, a small VM or a container on a schedule is usually simplest; for anything with dependencies, use the orchestrator.

## Time zones, the quiet bug

```
from datetime import datetime, timezone, timedelta

utc_now = datetime(2025, 12, 18, 2, 30, tzinfo=timezone.utc)
ist = utc_now.astimezone(timezone(timedelta(hours=5, minutes=30)))
print("server (UTC):", utc_now.strftime("%Y-%m-%d %H:%M"))
print("India (IST): ", ist.strftime("%Y-%m-%d %H:%M"))
print("same calendar date?", utc_now.date() == ist.date())
```

```
server (UTC): 2025-12-18 02:30
India (IST): 2025-12-18 08:00
same calendar date? True
```

A server in UTC running "at 02:30" is running at 08:00 in India, and a job that asks for "yesterday" gets a different answer depending on which clock it asks. Three habits fix it permanently:

1. **Set the schedule in the business's time zone**, and write the intended local time in a comment.
2. **Compute the reporting date explicitly** (`date.today()` in the business time zone, or pass it as an argument), never implicitly.
3. **Put the period in the subject line and the body**, so a reader can see which day they're looking at.

---

## 20.8 Alerts and exception reports

A daily report says what happened. An **alert** says something needs attention. They are different products, and mixing them is why people stop reading both.

|                    | Report             | Alert                       |
|--------------------|--------------------|-----------------------------|
| Arrives            | On a schedule      | When a condition is met     |
| Contains           | The full picture   | One problem, and what to do |
| Success looks like | Read in 20 seconds | Rare, and acted on          |
| Failure mode       | Ignored            | Ignored, which is worse     |

### Designing an exception rule

The Flash's rule is simple, which is a feature: *any product that sold fewer than 500 units today*. Here it is in pandas, on the real data:

```
import pandas as pd
from sqlalchemy import create_engine, text

engine = create_engine("postgresql+psycopg://book:book@localhost:5432/riverstone_full")
day = "2025-12-18"

lines = pd.read_sql(text("""
 SELECT p.product_name, s.quantity, s.net_revenue
 FROM sales_lines s JOIN products p ON p.product_id = s.product_id
 WHERE s.order_date = :day
"""), engine, params={"day": day})

by_product = lines.groupby("product_name", as_index=False)
["quantity"].sum().sort_values("quantity")
low = by_product[by_product["quantity"] < 500]

print(by_product.to_string(index=False))
print("\nexceptions:", len(low), "→", low["product_name"].tolist())
```

| product_name       | quantity |
|--------------------|----------|
| Industrial Crate   | 345      |
| Storage Box 25L    | 575      |
| Water Bottle 1L    | 600      |
| Lunch Box Set      | 790      |
| Food Container Set | 855      |
| Storage Box 10L    | 1005     |
| Stackable Bin      | 1240     |

exceptions: 1 → ['Industrial Crate']

One product falls below the line, so the email carries one exception box. On a day when nothing does, the email says so in green rather than showing an empty table: **silence is ambiguous, and an empty box looks like a bug**.

## Thresholds that survive contact with reality

- **Base them on history, not a round number.** "Below 500 units" is a placeholder; "below the 10th percentile of the last 90 days for that product" adapts as the business grows.
- **Require persistence** for noisy measures: alert when a threshold is crossed two days running, not on a single dip.
- **Add a floor for materiality:** don't alert on a ₹4,000 shortfall because it's 40% below target.
- **Say what to do.** "Storage Box 25L sold 345 units (below 500). Check stock at Bhiwandi and confirm the Mumbai dispatch." An alert without an action is only anxiety.
- **Count them.** If an alert fires every day, it's a report. If it never fires, nobody will believe it when it does; test it deliberately.

## Alert fatigue

The failure mode of alerting is volume. Three defences: **one rule, one owner; a weekly digest for anything not urgent**; and **a monthly review of every alert** asking "did anyone act on this?" Alerts nobody acted on get deleted, not tuned.

## 20.9 Delivering to chat

Where teams live in Teams, Slack, or WhatsApp, a message there beats an email nobody opens.

```
import os, requests

def post_to_webhook(text_summary, url_env="TEAMS_WEBHOOK_URL"):
 """Post a short summary to a Teams or Slack incoming webhook."""
 payload = {"text": text_summary} # Slack and Teams both accept a simple text
 payload
 response = requests.post(os.environ[url_env], json=payload, timeout=20)
 response.raise_for_status()
 return response.status_code
```

- **Incoming webhooks** are the simplest route: the channel owner creates a URL, and anything that can POST JSON can send to it. Treat the URL as a secret.
- **Richer formats:** Slack's Block Kit and Teams' Adaptive Cards let you send tiles and buttons. Keep the same discipline as email: a headline, a few numbers, and a link.
- **Link, don't dump.** Chat is for the headline and the exception; the detail lives in the report or the dashboard.
- **WhatsApp Business API** is common in Indian sales teams and is not a webhook: it needs an approved provider, pre-approved message templates, and consent. Check the rules before promising it; personal WhatsApp automation breaks the terms of service.
- **Threads and mentions:** mention a person only when you need them to act. A daily @channel is how a channel gets muted.

## 20.10 Low-code automation

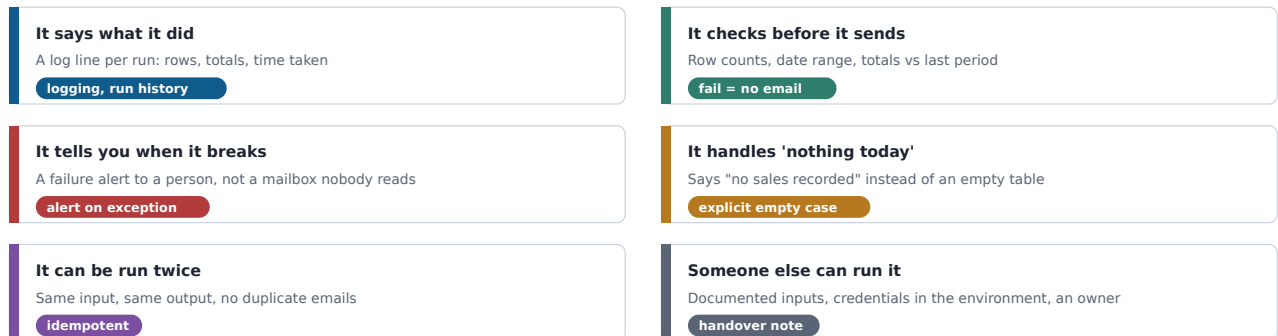
| Tool                              | Home ground          | Strengths                                                         | Watch for                                   |
|-----------------------------------|----------------------|-------------------------------------------------------------------|---------------------------------------------|
| <b>Power Automate</b>             | Microsoft 365        | Excel, Outlook, SharePoint, Teams, approvals; runs Office Scripts | Licensing tiers; flows owned by individuals |
| <b>n8n</b>                        | Self-hosted or cloud | Open source, code steps when needed, cheap at volume              | You maintain it                             |
| <b>Make</b> (formerly Integromat) | Cloud SaaS           | Visual, many connectors, good error handling                      | Per-operation pricing                       |
| <b>Zapier</b>                     | Cloud SaaS           | The widest connector list, easiest start                          | Costs rise quickly with volume              |

Low-code is the right answer when the work is **moving things between SaaS tools** (a form response becomes a CRM record becomes a Teams message), when **the business should own it**, or when **IT won't let you run scripts**. It's the wrong answer when the logic is complex, when it needs real data transformation, or when you'd be building a pipeline one box at a time: a flow with forty steps and no version control is harder to maintain than fifty lines of Python.

A pattern that works well in practice: **Python does the data work and writes a result somewhere** (a table, a file, an API endpoint), and **a flow does the delivery** (email, Teams, approvals). Each tool does what it's good at, and the business can change the recipients without touching code.

## 20.11 Making an automation trustworthy

### What makes an automation trustworthy



A report people trust is one they can check: the numbers, the time it ran, and what it excluded.

Figure 20.4 – The six properties that separate an automation people rely on from one they quietly stop believing.

### Checks that can stop it

Chapter 18's monthly report ran five checks before writing anything. The Flash does the same with the day's data:

```
from datetime import date

def checks(lines, day, previous_days):
 """Return (name, passed, detail) for each check. Any failure means no email."""
 revenue = float(lines["net_revenue"].sum())
 median_recent = float(previous_days["net_revenue"].median())
 return [
 ("rows returned", len(lines) > 0, f"{len(lines):,} lines"),
 ("all rows are today's", bool((lines["order_date"].dt.date == day).all()), str(day)),
 ("no missing revenue", bool(lines["net_revenue"].notna().all()), f"
{int(lines['net_revenue'].isna().sum())} missing"),
 ("revenue within 60% of recent median", abs(revenue / median_recent - 1) < 0.6,
 f"₹{revenue:,.0f} vs median ₹{median_recent:,.0f}"),
]

lines = pd.read_sql(text("""
 SELECT s.order_date, s.order_id, s.net_revenue FROM sales_lines s WHERE s.order_date = :day
"""), engine, params={"day": day}, parse_dates=["order_date"])
previous = pd.read_sql(text("""
 SELECT s.order_date, SUM(s.net_revenue) AS net_revenue FROM sales_lines s
 WHERE s.order_date BETWEEN :start AND :end GROUP BY s.order_date
"""), engine, params={"start": "2025-12-04", "end": "2025-12-17"}, parse_dates=["order_date"])

for name, passed, detail in checks(lines, date(2025, 12, 18), previous):
 print(f'{"PASS" if passed else "FAIL"} {name} ({detail})')
```

```
PASS rows returned (213 lines)
PASS all rows are today's (2025-12-18)
PASS no missing revenue (0 missing)
PASS revenue within 60% of recent median (₹2,511,819 vs median ₹3,209,520)
```

A check that fails is not a disaster; a check that doesn't exist is. The rule: **if a check fails, nothing is sent, and a person is told.**

## Logging and run history

```
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
datefmt="%H:%M:%S")
log = logging.getLogger("flash")
log.info("loaded %s lines for %s", len(lines), day)
log.warning("1 product below threshold")
print("(log lines go to stderr; the report goes to stdout, so a scheduler can capture them
separately)")
```

```
(log lines go to stderr; the report goes to stdout, so a scheduler can capture them separately)
```

Write a log line per run with the numbers that matter: rows read, total, exceptions raised, recipients, seconds taken. Then a run that produces a strange number can be explained a month later. Keeping the log in a table (a `run_history` table, or a sheet) also gives you the previous-run comparison for the checks.

## Failure alerts, "no data today", retries, and idempotency

- **Failure alert:** wrap the job in `try/except`, and on failure email a person ( `FLASH_FAILURE_TO` ) with the traceback, then exit non-zero so the scheduler also knows. A job that fails silently is worse than no job.
- **"No data today":** decide deliberately. For a daily sales report, send a short "no sales were recorded" note: a missing email is indistinguishable from a broken job.
- **Retries:** transient failures (a database restarting, a mail server timing out) deserve two or three retries with a growing pause. Permanent failures (a missing column) should not be retried; they should alert.
- **Idempotency:** running the job twice must not send two emails or double-load a table. Use a run key ( `flash_2025-12-18` ), record it, and exit early if it's already there.
- **Timeouts:** every network call gets one (Chapter 18, section 18.14), or one slow API holds the job forever.

```
run_key = f"flash_{day}"
already_sent = {"flash_2025-12-17"} # in a real job this comes from a table or a file
if run_key in already_sent:
 print(f"{run_key} already sent; exiting without sending again")
else:
 print(f"{run_key} not sent yet; proceeding")
```

## 20.12 Recipients and confidentiality

Most reporting incidents aren't wrong numbers. They're right numbers sent to the wrong people.

- **Keep the recipient list out of the code**, in a table or a config file with an owner and a review date. Lists made of individual addresses go stale; a distribution group maintained by IT doesn't.
- **Review the list quarterly**. People leave, change roles, and stay on lists for years.
- **Think about who sees what**. If branch managers shouldn't see each other's margins, either send per-branch emails from filtered data, or move to a dashboard with row-level security (Chapter 16, section 16.10). Never rely on a reader not scrolling.
- **Use To for people who must act and Cc sparingly**. Use Bcc for lists of external recipients so addresses aren't shared.
- **Attach deliberately**. The classic incident is an attachment with a hidden sheet, an unfiltered pivot cache, or last month's tab still in the workbook. If you attach an Excel file, generate it fresh from the data, don't reuse a template with old numbers inside.
- **Know what's sensitive**. Customer names, prices, margins, and salaries are not equally shareable. Ask before automating a report that contains any of them, and write the answer into the handover note.
- **Have a stop switch**. A `PAUSED=true` setting, or a single `enabled` flag in the config, so anyone can stop the sending without editing code.

## 20.13 Documenting and handing over

An automation is not finished when it runs; it's finished when someone else can own it. One page, kept with the code:

1. **What it does, in one sentence**, and who asked for it.
2. **When it runs**, in which time zone, on which machine, under which account.
3. **Inputs**: sources, tables, credentials (where they live, not what they are), parameters.
4. **Outputs**: what is produced, where it's written, who receives it.
5. **Checks**: what must pass before anything is sent, and what a failure means.
6. **Failure playbook**: the three most likely failures, how they look, and what to do. ("Refresh failed: the ERP load ran late. Rerun after 07:30 with `python daily_flash.py <date> --send .`")
7. **Owner and deputy**, with the date of the last review.

Two more habits make handover real: **run through it with the deputy once**, and **put the code in version control** (Chapter 26) rather than in a folder called `final_v3`.

## 20.14 Measuring what it saved

Automation is one of the few analyst activities with a simple business case, and analysts routinely fail to make it.

```
minutes_before, minutes_after, runs_per_year = 80, 2, 250
hours_saved = (minutes_before - minutes_after) * runs_per_year / 60
print(f"{hours_saved:,.0f} hours a year, about {hours_saved/8:,.0f} working days")
print(f"At a fully-loaded ₹1,200 an hour, that is ₹{hours_saved*1200:,.0f} a year")
```

```
325 hours a year, about 41 working days
At a fully-loaded ₹1,200 an hour, that is ₹390,000 a year
```

Record three things when you automate something: the time it took before (measured, not guessed), the time it takes now, and the errors it prevents. Then report it once a quarter, with the same discipline you'd apply to any other number. It's how the next automation gets approved, and how the work becomes visible to people who only see the output.

And keep a list of what to automate next, ordered by  $(\text{time saved} \times \text{frequency}) \div \text{effort}$ . The top of that list is rarely the most interesting problem, which is exactly why it's worth writing down.

## Common mistakes and how to spot them

| Mistake                               | Symptom                                               | Fix                                                 |
|---------------------------------------|-------------------------------------------------------|-----------------------------------------------------|
| Automating an unstable manual process | The script breaks every month                         | Stabilize at rung 2 first                           |
| Scheduling before the source is ready | Yesterday's numbers, silently                         | Check the load time; validate the date range        |
| The answer in an attachment           | Low open rates, "can you tell me the number instead?" | Put the headline in the subject and the body        |
| Modern CSS in an email                | Broken layout in Outlook                              | Tables, inline styles, 600-640 px                   |
| Images without alt and width          | A blank box in half the clients                       | Always set both; the email must work image-free     |
| Credentials in the script             | A password in Git forever                             | Environment variables, .env outside version control |
| Sending from a personal mailbox       | It stops when you leave or change your password       | A service account or shared mailbox                 |
| Scheduled on a laptop                 | Fails on leave, on updates, on sleep                  | A server, VM, container, or cloud scheduler         |
| Ignoring time zones                   | "Yesterday" differs from the reader's yesterday       | Schedule in business time; pass the date explicitly |
| No checks                             | A wrong report, delivered on time                     | Checks that stop the send                           |
| No failure alert                      | Nobody notices for three weeks                        | Alert a person, and exit non-zero                   |

| Mistake                      | Symptom                             | Fix                                              |
|------------------------------|-------------------------------------|--------------------------------------------------|
| Silence when there's no data | Indistinguishable from a broken job | Send "no sales were recorded"                    |
| Not idempotent               | Two emails, or double-loaded data   | A run key, recorded and checked                  |
| Alerts that fire daily       | Everyone mutes them                 | Tune thresholds; use persistence and materiality |
| Alerts with no action        | Anxiety, no decision                | Say what to do and who owns it                   |
| Recipient list in the code   | Stale lists; wrong people           | A config table with an owner and review date     |
| The wrong attachment         | A real confidentiality incident     | Generate fresh; never reuse a template with data |
| No documentation             | "Only Meera knows how it works"     | The one-page handover note                       |
| Never measuring              | The work is invisible               | Time before, time after, errors prevented        |

## In the real world: the flash that stopped

Riverstone's Daily Sales Flash went live on a Monday in February 2026: 07:30 IST, a service account, fourteen recipients, KPI tiles in the body. Vikram Singh stopped asking for the morning numbers on WhatsApp within a week, which was the point.

On 4 March, the Flash arrived showing ₹0 revenue and zero orders, with the green line saying no products were below threshold. The exception logic had nothing to look at; the checks had not been written yet; the email was technically correct and completely useless. Three managers rang the analytics team in fifteen minutes, which was the fastest feedback Meera had ever received about a report.

The cause was dull: the overnight ERP load had failed at 02:10, and the Flash read an empty table at 07:30.

She changed four things that afternoon.

1. **Checks before sending.** Rows returned, all rows are today's, no missing revenue, and revenue within 60% of the recent median. On 4 March, three of the four would have failed.
2. **A "no data" branch.** If there are no rows, the email says "No sales were recorded on 4 March. If that looks wrong, check the overnight load", and the subject says so too. It's short, honest, and unmistakable.
3. **A failure alert to a person.** Any exception, or any failed check, emails the analytics team with the traceback and exits non-zero, so the scheduler's own alerting fires as well.
4. **A dependency, not a clock.** The Flash now waits for a row in a small `load_status` table saying the ERP load finished, retrying every ten minutes until 08:30, and alerts if it never arrives.

The last change is the one that mattered most, and it's the one people skip: a report scheduled at a time is guessing; a report that waits for its input knows.

Two months later, the same mechanism caught something better. On 12 May the Flash's median check failed: revenue was 71% below the recent median. Not a broken load this time — a branch had entered a day of orders against the wrong month, which would have made May look terrible and April suspiciously good. The check held the email, the analytics team looked, the branch fixed it before 10 a.m., and nobody outside three people ever knew.

What made the difference:

- **The failure was visible to the machine first**, and the machine told a person.
- **Silence was never an option**: either the report, or a clear statement that there was nothing to report.
- **The checks encoded what "wrong" means**, so they caught a data problem the author hadn't imagined.
- **It waited for its input** instead of trusting the clock.

---

## Tools

- **Python 3.13 or 3.14** with `pandas`, `matplotlib`, `SQLAlchemy`, a database driver, `python-dotenv`, and `requests`. Everything runnable in this chapter was executed on Python 3.12 with `pandas 3.0.2`, `matplotlib 3.10.8`, `SQLAlchemy 2.0.54`, `psycpg 3.3.5`.
- **A mail route you're allowed to use**: SMTP with a service account, Microsoft Graph, the Gmail API, Apps Script's `MailApp` (Chapter 19), or a transactional provider.
- **A scheduler**: Windows Task Scheduler, cron, GitHub Actions, or a cloud scheduler.
- **Optional**: Power Automate, n8n, Make, or Zapier for delivery; Teams or Slack incoming webhooks.
- **Companion files ( companion/ch20/ )**: `daily_flash.py` — the complete Daily Sales Flash: query, headline calculations, exception rule, 14-day chart, HTML builder, SMTP sender, failure alert, "no data" branch, logging, arguments, and exit codes. It writes the rendered email to `out/daily_flash_<date>.html`, which is the file shown in Figure 20.3.

**Note on what was run.** The data work, the HTML, the chart, and the exception and check logic were executed against the real `riverstone_full` database, and Figure 20.3 is a screenshot of the actual email the script produced. The parts that need an account someone else controls — sending by SMTP or Graph, Teams and Slack webhooks, Task Scheduler and cron entries — are written out but could not be executed here, and are listed as manual checks.

---

## The project: Riverstone's Daily Sales Flash

**Goal:** an email that makes the morning call unnecessary, an exception alert when something is wrong, and a failure alert when the job breaks.

**Option A: your own report.** Take the report you produce most often and put it on rungs 2 and 3 of the ladder.

**Option B: Riverstone.** Build `daily_flash.py` yourself before reading the companion version.

### Steps

1. **Map the flow** and time the manual steps for a week. Write down what you'll automate and what stays judgment.
2. **One query** for the day's lines, one for the last 14 days, one for the same day last year.
3. **Headline numbers:** net revenue, orders, customers, average order value, gross margin, month to date, and the comparison with last year.
4. **Four checks that can stop the send**, including one that compares today with the recent median.
5. **The exception rule:** products below a threshold, with the threshold in one named constant and a plan to base it on history later.
6. **The email:** subject line with the headline, four KPI tiles, a 14-day chart, a segment table, the exception box (or the green "none today" line), and a footer saying where it came from.
7. **Sending:** SMTP or your workspace's API, credentials from the environment, recipients from config, `--send` off by default so you can look at the HTML first.
8. **Failure handling:** try/except around everything, an alert to a person, a non-zero exit code, and a log line per run.
9. **Schedule it** at 07:30 business time, with retries, and make it wait for (or check) the source load.
10. **Write the handover page** and the hours-saved calculation.

**What good looks like (18 December 2025):** net revenue ₹2,511,819, 118 orders from 116 customers, average order ₹21,287, gross margin 27.3%, +2.3% on the same day last year, month to date ₹5.71 crore, and one exception (Industrial Crate, 345 units). Running it twice sends one email.

### Stretch goals

- Add the formatted workbook from Chapter 18 as an attachment for the two people who want it, and nobody else.
- Send each branch manager only their branch's numbers, from the same script.
- Post a two-line summary to a Teams or Slack channel at the same time, linking to the Power BI report from Chapter 16.
- Store each run's headline numbers in a `run_history` table, and use it for the "versus recent median" check instead of recomputing.

---

## Timed challenge: forty minutes

---

Use `riverstone_full` and Python. Answers at the end of the chapter.

- **Level 1:** For 18 December 2025, compute net revenue, orders, customers, and average order value.

- **Level 2:** Compute the same day's gross margin percentage, and the comparison with 18 December 2024.
  - **Level 3:** Month-to-date revenue to 18 December 2025.
  - **Level 4:** The 14-day daily series ending that day: which day was highest, and which lowest?
  - **Level 5:** Products selling fewer than 500 units that day.
  - **Level 6:** Write the subject line your script would send, with the headline number and the comparison.
  - **Level 7:** Write the four checks and run them for 18 December 2025 and for a date with no data (1 January 2026). What happens in each case?
  - **Bonus:** Compute the hours saved a year if the manual version took 80 minutes and the automated one takes 2, at 250 runs a year.
- 

## You've got it when...

---

- ☐ You can place a report on the ladder and say why it should stop there.
  - ☐ You map a flow and time the steps before writing code.
  - ☐ You choose the delivery tool from where the data and the maintainer are, not from preference.
  - ☐ You put the answer in the subject line and the body, and attach only what's actually needed.
  - ☐ You can build an HTML email that renders in Outlook: tables, inline styles, 640 px, `alt` text.
  - ☐ You send mail with credentials from the environment, from an account that isn't personal.
  - ☐ You schedule in business time, pass the reporting date explicitly, and know why cron needs absolute paths.
  - ☐ Your alerts have thresholds with a rationale, an action, and an owner, and you review whether anyone acts on them.
  - ☐ Your automations log every run, check before sending, alert a person on failure, say "no data today", and can be run twice safely.
  - ☐ Your recipient list lives in config with a review date, and you've thought about who may see what.
  - ☐ There's a one-page handover note, and someone else has run the job.
  - ☐ You can state the hours the automation saves and the errors it prevents.
- 

## Recap

---

- **The ladder** runs manual → refreshable → scheduled → triggered → self-serve. Stabilize before scheduling; everything above rung 2 needs an owner, a log, and a failure alert.
- **Map the flow and time it** before automating. Automate the mechanical steps, keep the judgment, and give the judgment a place in the output.

- **Choose the tool** from where the data lives and who will maintain it: VBA, Apps Script, Python, BI subscriptions, or a low-code flow.
- **Put the answer in the email body**, with the headline in the subject. Attachments are for people who asked for them.
- **Email HTML is old HTML**: tables, inline styles, 600–640 px, web-safe fonts, images with `alt` and `width`, no JavaScript. CID images are the reliable route for Outlook.
- **Send from a service account**, with credentials in the environment, through SMTP, Graph, Gmail, or a provider.
- **Schedule in business time**, pass the date explicitly, use absolute paths in cron, and prefer waiting for the source load over trusting the clock.
- **Alerts are not reports**: thresholds based on history, persistence, materiality, an action, an owner, and a periodic review of whether anyone acted.
- **Trustworthy means**: it logs, it checks before sending, it alerts a person when it breaks, it handles "no data today", it can run twice, and someone else can run it.
- **Recipients and confidentiality** are where real incidents happen: config-driven lists, quarterly review, per-recipient filtering or row-level security, deliberate attachments, and a stop switch.
- **Document and hand over**, then **measure**: 80 minutes to 2, 250 times a year, is 325 hours.

---

## Practice exercises

---

Use `riverstone_full`, `companion/ch20/daily_flash.py`, and your own email account only where it's safe.

### Warm-up

1. Place each on the ladder and say what it would take to move it up one rung: a weekly stock count pasted into a workbook; a Power BI report with a subscription; a monthly pack built by a macro; an ad-hoc pricing analysis.
2. A report is scheduled at 06:00 and the source table loads at 06:30. What are three ways to fix it, and which is best?
3. Why does an attachment lower the chance a report is read? When is an attachment still the right choice?
4. Name four things that break HTML email layout, and the safe alternative for each.
5. Where should database and SMTP credentials live, and what's wrong with each of these: in the script; in a config file committed to Git; in the scheduler's command line?
6. What's the difference between a report and an alert, and what happens when you mix them?

### Core

7. Run `daily_flash.py` for 18 December 2025 without `--send` and open the HTML. What are the four KPI values?

8. Change the exception threshold to 800 units and rerun. How many exceptions appear, and which products?
9. Run it for 1 January 2026 (a date with no data). What does it produce, and why is that better than sending nothing?
10. Add a fifth KPI tile showing the number of order lines, and check that the row of tiles still fits at 640 px.
11. Write the four checks from section 20.11 as a function and run them for 18 December 2025. What does each report?
12. Deliberately break a check (for example, require revenue within 5% of the median) and confirm that nothing is sent and the log says why.
13. Add a `run_history` table (or a CSV) that records the date, revenue, exceptions, and run time. Use it to make the job idempotent.
14. Rewrite the exception rule to use the 10th percentile of the last 90 days for each product instead of a fixed number. Which products would be flagged on 18 December 2025?
15. Write the subject line generator, including the "no data" case, and test it for both dates.
16. Add a plain-text alternative to the email and confirm the message has both parts.
17. Write the Task Scheduler or cron entry for 07:30 on weekdays, including the working folder and log redirection.
18. Write a `post_to_webhook` function for Teams or Slack and a two-line summary message. (Test it only against a channel you own.)
19. Draft the one-page handover note for the Flash, using the seven headings in section 20.13.
20. Calculate the hours saved for a report in your own work, and write the two sentences you'd put in a quarterly update.

## Stretch

21. Send each branch manager only their branch's numbers from one run, and explain how you'd verify that nobody received another branch's figures.
22. Make the job wait for a `load_status` row instead of trusting the clock, with retries every ten minutes until 08:30 and an alert if it never arrives.
23. Add a weekly digest on Mondays that summarizes the week's exceptions, so daily alerts can be reserved for what needs same-day action.
24. Rebuild the delivery half in a low-code tool (Power Automate, n8n, or Make) with Python producing the data, and compare maintainability.

## Think about it

25. Your Flash has run for six months. How would you find out whether anyone still reads it, and what would you do if the answer is "two people"?
26. A manager asks for a daily alert whenever any customer's order is 20% below their average. What do you ask before building it?
27. An automation you built emails a confidential margin report to a list that turns out to include a contractor. What do you do in the first hour, and what do you change afterwards?

---

## Key terms

---

automation ladder · refreshable · scheduled · triggered · self-serve · report flow map · delivery tool · formatted Excel · PDF · CSV extract · HTML email · KPI tile · inline styles · table layout · web-safe font · alt text · data URI · CID attachment · SMTP · STARTTLS · service account · shared mailbox · Microsoft Graph · Gmail API · transactional provider · Task Scheduler · cron · working directory · cloud scheduler · time zone · reporting date · alert · exception report · threshold · persistence · materiality · alert fatigue · incoming webhook · Adaptive Card · Block Kit · WhatsApp Business API · low-code · Power Automate · n8n · Make · Zapier · logging · run history · check · failure alert · exit code · "no data today" · retry · idempotency · run key · recipient list · row-level filtering · stop switch · handover note · hours saved

*(All terms are defined in the Glossary, Appendix A.)*

---

## Where this leads

---

- **Chapter 21 and 22:** the statistics behind sensible thresholds, and why a single day's dip usually isn't a signal.
  - **Chapter 23, Data Storytelling:** what to write in the three sentences of commentary you kept.
  - **Chapter 26, Git:** versioning automations, and keeping `.env` out of the repository.
  - **Chapter 30, Python as Software:** packaging, tests, and configuration once a script becomes a tool several people depend on.
  - **Chapter 46:** orchestration, when "run this at 7" becomes "run these eleven things in the right order, with retries".
  - **Chapter 47:** data-quality testing, which is the checks in this chapter done systematically.
  - **Interview preparation:** the Business Analyst bank (Chapter 76) asks how you'd automate and deliver a recurring report, and what you'd do when it fails.
- 

## Answers to practice exercises

---

*(In the finished book these move to Appendix G.)*

1. Weekly stock count: rung 1 → rung 2 by making the paste a Power Query refresh. Power BI subscription: rung 3 already; rung 4 would be a data-driven alert, rung 5 is self-serve, which it partly is. Monthly macro pack: rung 3 if scheduled, rung 2 if someone clicks the button; move it up by scheduling it somewhere that isn't a laptop. Ad-hoc pricing analysis: rung 1, and it should stay there.
2. (a) Move the schedule to 07:00 and hope; (b) check the data before reporting and fail if it's stale; (c) make the job wait for a "load finished" signal and retry. (c) is best, (b) is the minimum, and (a) is what most teams do until it bites.

3. An attachment needs a download, an app, and a second decision, and on a phone it's often impossible. It's right when the reader will work with the data (filter, pivot, paste into a model), or when a fixed record is needed for compliance.
4. Flexbox or grid → tables; `<style>` blocks or external CSS → inline styles; web fonts → web-safe fonts with fallbacks; wide layouts → 600–640 px; also images without `width / alt` → always set both.
5. In environment variables, loaded from a `.env` file that isn't committed, or a secret manager. In the script: they end up in Git and in everyone's copy. In a committed config file: the same problem with extra steps. On the scheduler's command line: visible in process lists and logs.
6. A report arrives on a schedule and shows the full picture; an alert arrives when something is wrong and names one problem and an action. Mixed, the alert becomes routine and gets ignored, and the report becomes noisy.
7. Net revenue **₹2,511,819** (+2.3% vs last year) · **118** orders from 116 customers · average order **₹21,287** with 27.3% margin · month to date **₹5.71 crore**.
8. At 800 units, four products are flagged: Industrial Crate (345), Storage Box 25L (575), Water Bottle 1L (600), and Lunch Box Set (790). That's the tuning problem in one experiment: the threshold decides whether the alert is useful or noise.
9. It writes the "no sales were recorded on 1 January 2026" email, with that in the subject too, and logs a warning. It's better than silence because a missing email is indistinguishable from a broken job, and better than an empty table because it tells the reader what to check.
10. Five tiles at 640 px are cramped; either drop to four, use two rows of two, or shorten the labels. The answer is a design decision, and the check is opening it on a phone.
11. On 18 December 2025: rows returned PASS (213 lines), all rows are today's PASS, no missing revenue PASS, revenue within 60% of the recent median PASS (₹2,511,819 against a median of ₹3,209,520).
12. With a 5% band the median check fails, the job logs `FAIL revenue within 5% of recent median`, writes no email, alerts the owner, and exits non-zero. That's the behaviour you want; the lesson is that too tight a threshold turns a safety net into a blocker.
13. Store `run_key`, date, revenue, exception count, and duration. At the start, `SELECT 1 FROM run_history WHERE run_key = :key`: if it exists, log and exit 0 without sending.
14. With a per-product 10th percentile of the last 90 days, the fixed-threshold flags change: products with structurally low volumes (Industrial Crate) stop being flagged every day, and a genuine drop in a normally high-volume product is caught. That's the point of a relative threshold.
15. `f"Riverstone Daily Flash – {day:%d %b %Y} – ₹{revenue:,.0f} ({vs_ly:+.1f}% vs LY)"`, and for the empty case `f"Riverstone Daily Flash – {day:%d %b %Y} – no sales recorded"`.
16. `message.set_content(...)` then `message.add_alternative(html, subtype="html")` gives a `multipart/alternative` message; check with `message.get_content_type()` and by viewing the source in the client.

17. cron: `30 7 * * 1-5 cd /opt/riverstone && /opt/riverstone/.venv/bin/python daily_flash.py "$(date +%F)" --send >> logs/flash.log 2>&1` . Task Scheduler: action = the venv's `python.exe` , arguments = `daily_flash.py %date% --send` , **Start in** = the script folder, trigger daily 07:30 weekdays, with restart on failure.
18. See section 20.9. The summary should be two lines: the headline number with the comparison, and the exception count, plus a link to the dashboard.
19. The seven headings from section 20.13, filled in for the Flash: what and who asked; 07:30 IST weekdays on the reporting VM under the `svc-analytics` account; sources and credentials; outputs and recipients; the four checks; the three likely failures with commands to rerun; owner and deputy with a review date.
20. For example: *"Automating the daily flash replaced 80 minutes of manual work with 2, 250 times a year: about 325 hours, or 40 working days. It has also caught two data errors before they reached managers."*
21. Loop the branches, filter the data, and send one email per branch, with the branch in the subject. Verify by logging, for each send, the recipient and a hash or total of the data sent, then spot-check three; and by sending the first run to yourself with the branch name in the body.
22. Poll the `load_status` table every ten minutes from 07:30; if the row appears, run; if 08:30 passes without it, send a failure alert naming the missing load. Log each attempt so a late load is visible afterwards.
23. A Monday job that reads `run_history` and the exception log for the previous week, groups by product and branch, and sends one digest. Daily alerts then carry only same-day actions, which is what keeps them credible.
24. Python writes the day's numbers to a table or a JSON file; the flow reads it, formats the message, and sends it. Maintainability improves for the business (they can change recipients and wording) and worsens for you (logic split across two places): document where the boundary is.
25. Ask the mail platform for open rates if you have them, put a tracked link to the dashboard in the footer and watch the clicks, or stop sending it for a week and see who asks. If two people read it, either narrow it to those two, fold it into a weekly digest, or replace it with an alert; a report read by two people is not a failure if those two make decisions with it.
26. What decision follows the alert? How many customers, and therefore how many alerts a day? What's "average": mean or median, over what period? Does a 20% drop matter for a ₹5,000 customer? Who acts, and by when? And would a weekly list of the twenty biggest declines serve better than daily alerts?
27. First hour: stop the schedule (the stop switch), find out exactly what was sent and to whom, tell your manager and whoever owns data protection, and don't try to recall the email quietly. Afterwards: recipient lists in config with an owner, a review date, and a check that every recipient is on an approved domain; a test send to yourself on every change; and sensitivity written into the handover note.

**Timed challenge answers.** Level 1: ₹2,511,819 · 118 orders · 116 customers · ₹21,287 average order. Level 2: 27.3% gross margin; +2.3% against 18 December 2024's ₹2,454,466. Level 3: **₹57,069,985** (₹5.71 crore) from 1 to 18 December. If you get ₹4.45 crore, your window started on the 5th, not the 1st: reusing the 14-day series for month to date is the most common slip, and it was a real bug in the first version of `daily_flash.py`. Level 4: highest **7 December (₹3,782,009)**, lowest **9 December (₹2,488,217)**; the 18th itself is ₹2,511,819, a little above the low. Level 5: Industrial Crate, 345 units. Level 6: Riverstone Daily Flash – 18 Dec 2025 – ₹2,511,819 (+2.3% vs LY). Level 7: all four pass for 18 December 2025; for 1 January 2026 the first check fails, nothing is sent, and the "no data" email goes out instead. Bonus: 325 hours, about 40 working days.